EE 446: TinyML

Lab 5 — Key Word Spotting with TinyML

Student Information

Full Name: Luke Valerio

Student ID: 1901711

1. The purpose of converting raw audio into spectrogram/MFCC-style features

A raw 1-second audio clip at 16 kHz contains 16,000 amplitude samples. In this raw form, there is no explicit structure. The crucial information that distinguishes spoken words, like which frequencies are present and how they shift over time, gets completely buried in that time-domain signal.

Converting the audio into a spectrogram or MFCC-style representation brings that hidden structure to the surface. We break the signal into short, overlapping windows and use a short-time Fourier transform to measure the energy in each frequency band over time. This helps the model in three main ways:

- **Dimensionality reduction:** The bulky 16,000-sample waveform shrinks down into a small 2D feature map (around 1,960 values). This is much cheaper and easier for a tiny microcontroller to process.
- **Perceptually meaningful features:** The Mel scale spaces out the frequency bins similarly to how human hearing works, putting the focus right on the bands where speech information actually lives.
- **Robustness:** The model gets to look at the overall pattern of energy across frequency and time. This pattern stays much more stable across different speakers, volume changes, and minor timing shifts compared to the raw waveform, which can look wildly different even for the exact same spoken word.

In short, this process hands the neural network exactly the low-dimensional, highly distinguishing information it needs to learn and recognize keywords reliably.

2. The purpose of the representative dataset in INT8 quantization

Full integer (INT8) quantization works by replacing a model's 32-bit floating-point weights and activations with much smaller 8-bit integers. Since the model's weights are fixed, we can easily measure their range. However, activations (the intermediate data produced as inputs flow through the network) depend entirely on what data you feed it, meaning we do not know their range ahead of time.

To properly map each floating-point tensor to an INT8 range of [-128, 127], the converter needs to know the actual minimum and maximum range of every activation. This is exactly what the representative dataset provides. It is a small batch of example inputs (in this notebook, 100 samples from the test partition) that the converter runs through the model. By doing this, it can observe the real-world distribution of activation values at each layer.

Once it observes those ranges, it calculates the specific scale and zero-point needed to quantize each tensor. This step is critical. If we just guessed the ranges or set them poorly, the quantization process would either clip important data or waste precision, causing the model's accuracy to drop sharply. Because of this, your representative data needs to closely resemble the real inputs the model will see in production so the calibration is accurate. This calibration is absolutely essential for creating a fully integer model, which is ultimately what allows it to run on an integer-only microcontroller like the Arduino Nano 33 BLE. It uses roughly four times less memory and benefits from much faster integer arithmetic, all while keeping accuracy very close to the original floating-point model.

3. What audio_processor.get_data() does

The `get_data()` method belongs to the `AudioProcessor` class. Its main job is to assemble a batch of preprocessed feature vectors along with their corresponding integer labels from the Speech Commands dataset.

When you pass it a specific partition mode like training, validation, or testing, it selects the appropriate audio clips. For training, it grabs a random selection. For validation or testing, it deterministically grabs the first N clips so your metrics remain repeatable. Then, it runs each clip through the data pipeline and applies augmentation. It loads the WAV file, can optionally apply a random time shift, and can even mix in background noise. After that, it runs the exact same feature extraction you would use in production (the MFCC front end) to turn every 1-second clip into a ready-to-use input feature map for the model.

You can use the `how_many` argument to control batch size (setting it to -1 returns the entire partition) and `offset` to decide where to start fetching from. The method ultimately returns two things: a list of the transformed sample data and a matching list of label indices. In this specific workflow, it serves two distinct purposes. First, it supplies the representative dataset needed for INT8 quantization. Second, it fetches the full test set so you can evaluate the accuracy of both the floating-point and quantized models.

4. Why g_model and g_model_len must be copied correctly

`g_model` is a C array that holds the exact bytes of your quantized `.tflite` model (the FlatBuffer file), and `g_model_len` is an integer that simply tells you the total number of bytes in that array.

The final export step takes your binary model file and rewrites it as a plain text C source file, formatting each byte as a hex value. This is necessary because microcontrollers usually do not have a traditional filesystem to load a model from at runtime. Instead, the model has to be compiled directly into the firmware and live in the device's flash memory as a constant array.

Once on the Arduino, TensorFlow Lite Micro is given a pointer to `g_model` so it can build the interpreter, using `g_model_len` to know exactly how many bytes to read. It is incredibly important that these are copied correctly because the FlatBuffer is parsed strictly byte-for-byte. If a single byte is missing, extra, altered, or truncated, or if the declared length does not match the actual array size, the model gets corrupted. If that happens, the interpreter will either fail to initialize, misallocate memory, or just spit out garbage predictions. Put simply, for your deployed keyword spotting model to actually work, `g_model` must be the complete, unmodified model, and `g_model_len` must perfectly match its true byte count.

5. Include a screenshot showing correct identification of at least one keyword ("YES" or "NO") in the Arduino IDE Serial Monitor after deploying the KWS model on the Arduino Nano 33 BLE.

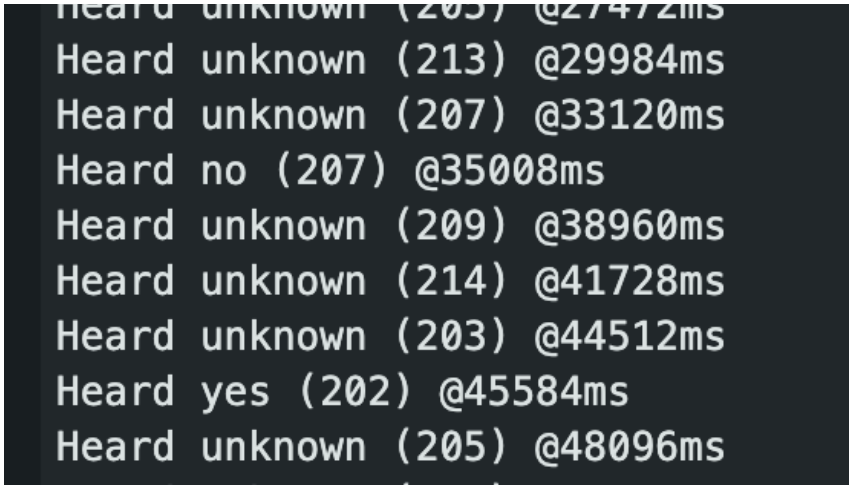

Figure 1. Arduino IDE Serial Monitor — correct detection of "no" and "yes".